

ماجرای جویی با پیتون

سفری پروژه محور به دنیای برنامه نویسی

نویسنده: محمد مهدی قائمی

نام و لوگوی انتشارات

:	سرشناسه
:	عنوان قراردادی
:	عنوان و نام پدیدآور
:	مشخصات نشر
:	مشخصات ظاهری
:	شابک
:	وضعیت فهرست نویسی
:	یادداشت
:	موضوع
:	موضوع
:	رده بندی کنگره
:	رده بندی دیویی
:	شماره کتاب شناسی ملی

ماجرای جویی با پایتون

نویسنده: محمد مهدی قائمی

ناشر: انتشارات

شمارگان: نسخه

نوبت چاپ: اول، ۱۴۰۴

قیمت: تومان

شابک: - - - ۹۷۸-۶۰۰-

سخن ناشر

مقدمه

به دنیای شگفت‌انگیز پایتون خوش آمدید!

قرار است در این کتاب، دست در دست هم از نقطه صفر شروع کنیم و قدم‌به‌قدم پایتون را یاد بگیریم؛ اما نه با درس‌های خشک و خسته‌کننده، بلکه با ساختن پروژه‌های جذاب و واقعی که مسیر یادگیری را برایتان شیرین و لذت‌بخش می‌کند.

انسان همیشه رویای داشتن یک «طلسم جادو» را در سر داشت؛ ابزاری که کارهای سخت و تکراری را به عهده بگیرد تا او بتواند به کارهای مهم‌تر بپردازد. از چرتکه‌های چوبی قدیمی تا ماشین‌حساب‌ها و در نهایت اختراع کامپیوتر، همه تلاشی برای رسیدن به همین رویا بودند: سرعت بالاتر و خطای کمتر. با آمدن کامپیوترها، حل مسائلی که زمانی غیرممکن به نظر می‌رسید، ممکن شد. اما یک مشکل بزرگ وجود داشت: زبان کامپیوترها عجیب و سخت بود! اینجا بود که «زبان‌های برنامه‌نویسی» متولد شدند تا پلی باشند میان زبان انسان و زبان ماشین.

در میان صدها زبانی که آمدند و رفتند، «پایتون» ستاره‌ای بود که درخشید. پایتون آنقدر ساده و روان است که یادگیری‌اش مثل آب خوردن است، و در عین حال آنقدر قدرتمند است که غول‌های فناوری دنیا برای ساخت هوش مصنوعی، تحلیل داده، وب‌سایت‌های بزرگ از آن استفاده می‌کنند.

به همین دلیل است که امروز پایتون محبوب‌ترین زبان دنیاست. میلیون‌ها نفر این مسیر را انتخاب کرده‌اند تا آینده‌ای هوشمند بسازند. حالا نوبت شماست؛ آماده‌اید؟

فهرست مطالب

بخش اول: شروع ماجراجویی! س

فصل ۱: اولین سفر: آماده‌سازی محیط توسعه ۱۷

چرا پایتون؟ ۱۸

آماده کردن کوله‌پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین ۱۹

مسیر اول: ساخت کارگاه شخصی (نصب پایتون روی کامپیوتر و لپ‌تاپ) ۲۰

قدم اول: نصب مترجم پایتون ۲۰

۱. برای کاربران ویندوز (Windows) ۲۰

۲. برای کاربران مک (The Mac Guild) ۲۰

۳. برای کاربران لینوکس (Linux) ۲۱

قدم دوم: آماده کردن دفترچه جادو (محیط توسعه VS Code) ۲۱

قدم سوم: افزونه شناسایی پایتون در دفترچه جادویی (VsCode) ۲۲

مسیر دوم: استفاده از آزمایشگاه جادویی آنلاین (Google Colab) ۲۲

اولین ورد جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه! ۲۳

پروژه: ساخت کارت ویزیت دیجیتال ۲۶

فصل ۲: تبدیل شدن به کارآگاه کد ۲۹

ذهنیت یک ماجراجو: حل معما، نه صرفاً حفظ کردن ورد! ۳۰

جعبه ابزار کارآگاه ۳۰

۱. Stack Overflow: کتابخانه بزرگ دانش تمام جادوگران دنیا ۳۰

۲. دستیارهای هوشمند (AI): غول‌های چراغ جادوی ۳۱

۳۲ پروژه: شکار باگ در کد نفرین شده

فصل ۳: جعبه های جادویی: ماشین حساب جادویی

۳۶ ۱. جعبه های جادویی (متغیرها)

۳۷ ۲. آشنایی با غنائم: انواع داده های پایه (Primitive Types)

۳۸ اعداد صحیح (Integer)

۳۸ اعداد اعشاری (Float)

۳۸ رشته (String)

۳۹ بولین (Boolean)

۳۹ ۳. جادوی ریاضی: کار با عملگرها

۴۱ ۵. پروژه: ساخت ماشین حساب جادویی

فصل ۴: منطق و شرطها: ساخت بازی حدس عدد

۴۴ ۱. صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()`

۴۴ ۲. تله ی کیمیاگری (تبدیل متن به عدد)

۴۵ ۳. دو راهی سرنوشت (`if` و `else`)

۴۶ ۴. مسیرهای چندگانه (`elif`)

۴۷ ۵. تکرار تا پیروزی (حلقه `while`)

۴۷ ۶. پروژه: بازی حدس عدد

بخش دوم: مدیریت کوله پشتی ماجراجو

فصل ۵: مدیریت هوشمند کوله پشتی: لیست خرید

Error! Bookmark not defined. ۱. کوله پشتی جادویی (Lists)

Error! Bookmark not defined. ۲. پر کردن کوله پشتی (اضافه کردن، دیدن و حذف کردن) ..
defined.

Error! Bookmark not defined. `append()`: اضافه کردن آیتم

Error! Bookmark not defined. خواندن آیتم ها با «ایندکس» (`Index`)

Error! Bookmark not defined. `remove()`: حذف کردن آیتم

۳. بازرسی کوله پشتی (طلسم‌های len و in)..... Error! Bookmark not defined.

ورد len(): طلسم شمارش‌گر Error! Bookmark not defined.

ورد in: طلسم جستجوگر Error! Bookmark not defined.

۴. سرشماری غنائم (حلقه for)..... Error! Bookmark not defined.

۵. پروژه: ساخت برنامه مدیریت لیست خرید..... Error! Bookmark not defined.

فصل ۶: تکنیک‌های پیشرفته کوله پشتی: تابلوی امتیازات
Error! Bookmark not defined.

۱. برش زدن کوله پشتی (Slicing)..... Error! Bookmark not defined.

۲. کیسه‌ی مهر و موم شده (Tuples)..... Error! Bookmark not defined.

۳. جادوی باز کردن تاپل (Tuple Unpacking)..... Error! Bookmark not defined.

۴. ترکیب جادوها: لیستی از تاپل‌ها..... Error! Bookmark not defined.

۵. پروژه: ساخت تابلوی امتیازات "Top 3"..... Error! Bookmark not defined.

فصل ۷: کتاب جادویی: ساخت مترجم کلمات
Error! Bookmark not defined.

۱. دیکشنری‌ها (Dictionaries): کتاب جادویی شما Error! Bookmark not defined.

۲. جادوی واژه‌نامه: افزودن، تغییر و حذف..... Error! Bookmark not defined.

اضافه کردن یا تغییر دادن یک «ورد»..... Error! Bookmark not defined.

حذف کردن یک «ورد»..... Error! Bookmark not defined.

۳. تله‌ی کلید گمشده (The KeyError)..... Error! Bookmark not defined.

۴. طلسم‌های ایمنی (in و get)..... Error! Bookmark not defined.

طلسم اول: بررسی با in..... Error! Bookmark not defined.

طلسم دوم (حرفه‌ای): متد get()..... Error! Bookmark not defined.

۵. سرشماری در کتاب جادویی (Looping)..... Error! Bookmark not defined.

۶. هشدار ماجراجو: طلسم نمایش حروف فارسی Error! Bookmark not defined.

۷. پروژه: ساخت مترجم کلمات..... Error! Bookmark not defined.

فصل ۸: ساخت جادوهای شخصی: هنر ساخت توابع
Error! Bookmark not defined.

۱. "یک بار بنویس، صد بار استفاده کن": معرفی توابع
Error! Bookmark not defined.

۲. «تعریف» (Define) و «صدا زدن» (Call) یک طلسم
Error! Bookmark not defined.

۳. اولین طلسم ما: خداحافظی با تکرار.....
Error! Bookmark not defined.

۴. ورودی و خروجی توابع (Arguments و return).....
Error! Bookmark not defined.

۵. پروژه: رفع طلسم «طومار تکراری».....
Error! Bookmark not defined.

فصل ۹: طلسم محافظتی: ساخت رمز عبور امن
Error! Bookmark not defined.

۱. استفاده از قدرت‌های مخفی پایتون: کتابخانه‌ها و import
Error! Bookmark not defined.

۲. کتابخانه random: بهترین دوست ما برای کارهای تصادفی ...
Error! Bookmark not defined.

۳. جادوی شمارش: تکرار با range() for i in
Error! Bookmark not defined.

۴. پروژه: انتخاب رمز برای صندوقچه گنج خودمان.....
Error! Bookmark not defined.

بخش سوم: ورود به بعد چهارم
Error! Bookmark not defined.

فصل ۱۰: رمزگشایی از طومارهای باستانی: تحلیلگر متن
Error! Bookmark not defined.

۱. کار حرفه‌ای با متن‌ها (طلسم‌های String).....
Error! Bookmark not defined.

طلسم اول: جادوی باستانی (قالب‌بندی با %).....
Error! Bookmark not defined.

طلسم دوم: جادوی مدرن (متد format()).....
Error! Bookmark not defined.

طلسم سوم: جادوی استاد (f-strings).....
Error! Bookmark not defined.

۳. پروژه: ساخت تحلیلگر متن.....
Error! Bookmark not defined.

فصل ۱۱: طلسم ضد فراموشی: کار با فایل ها Error! Bookmark not defined.

۱. چطور از شر "کرش" کردن خلاص شویم؟ (try و except). Error! Bookmark not defined.

not defined.

۲. خواندن و نوشتن در فایل های متنی (txt.). Error! Bookmark not defined.

۳. زبان جادویی جهانی (JSON). Error! Bookmark not defined.

۴. طلسم زبان های باستانی: جادوی encoding (مخصوص فارسی). Error!

Bookmark not defined.

۵. پروژه: ارتقاء «مدیر لیست خرید». Error! Bookmark not defined.

فصل ۱۲: دمیدن روح در کد: خلق موجودات با شیء گرایی Error! Bookmark not

defined.

۱. ترکیب داده و رفتار: کلاس (Class) و شیء (Object). Error! Bookmark not

defined.

۲. "نقشه ساخت" (کلاس) و طلسم خلقت (__init__). Error! Bookmark not

defined.

۳. ویژگی ها (Attributes): ذخیره داده ها روی self. Error! Bookmark not

defined.

۴. رفتارها (Methods): افزودن جادو به موجودات. Error! Bookmark not

defined.

۵. پروژه: ساخت مغازه جادوگری. Error! Bookmark not defined.

بخش چهارم: آیین های جادوگر ارشد Error! Bookmark not defined.

فصل ۱۳: سازماندهی کتابخانه جادویی (ساخت ماژول های شخصی) Error!

Bookmark not defined.

۱. چرا یک فایل کافی نیست؟ (اصل تفکیک مسئولیت ها). Error! Bookmark not

defined.

۲. طلسم import شخصی (ساخت اولین ماژول). Error! Bookmark not

defined.

۳. پروژه: معماری «مغازه جادوگری». Error! Bookmark not defined.

فصل ۱۴: کوله پشتی جادویی زمان Error! Bookmark not defined.

۱. احضار datetime..... Error! Bookmark not defined.

۲. طلسم now() (گرفتن لحظه حال)..... Error! Bookmark not defined.

۳. قالب‌بندی زمان (طلسم strftime)..... Error! Bookmark not defined.

۴. سفر در زمان (طلسم timedelta)..... Error! Bookmark not defined.

۵. پروژه: اعلان گر مهلت (The Deadline Notifier) Error! Bookmark not defined.

فصل ۱۵: آیین قبیله پایتون هنر کدنویسی پایتونیک Error! Bookmark not defined.

۱. ذن پایتون (The Zen of Python)..... Error! Bookmark not defined.

۲. طلسم enumerate (شمارش پایتونیک)..... Error! Bookmark not defined.

۳. طلسم zip (جفت کردن طومارها)..... Error! Bookmark not defined.

۴. طلسم تک‌خطی (List Comprehensions)..... Error! Bookmark not defined.

۵. جادوی ناشناس (Lambda)..... Error! Bookmark not defined.

۶. پروژه: کیمیاگری لیست (List Alchemy)..... Error! Bookmark not defined.

بخش پنجم: نبرد نهایی (پروژه بزرگ ماجراجو) Error! Bookmark not defined.

فصل ۱۶: نبرد نهایی: ساخت دفترچه مأموریت ماجراجو Error! Bookmark not defined.

۱. نگاهی به گذشته (Gathering Your Spells) Error! Bookmark not defined.

۲. گزارش شناسایی (The Adventurer's Challenge) ... Error! Bookmark not defined.

۳. میز معمار (Building Step-by-Step)..... Error! Bookmark not defined.

بخش ششم: ورود به تالار پیشگویان (مسیر دانشمند داده)
Error! Bookmark not defined.

فصل ۱۷: دروازه احضار و آزمایشگاه جادویی (Pip & Venv)
Error! Bookmark not defined.

۱. آزمایشگاه ایزوله (venv) Error! Bookmark not defined.

۲. فعال‌سازی آزمایشگاه (ورود به حباب) Error! Bookmark not defined.

۳. دروازه احضار (pip) Error! Bookmark not defined.

۴. طومار نیازمندی‌ها (requirements.txt) Error! Bookmark not defined.

۵. پروژه: تابلوی اعلانات جادویی Error! Bookmark not defined.

فصل ۱۸: طومارهای NumPy (جادوی محاسبات نورآسا)
Error! Bookmark not defined.

۱. احضار NumPy Error! Bookmark not defined.

۲. ndarray چیست؟ (ارتش منظم) Error! Bookmark not defined.

۳. جادوی Vectorization (حذف حلقه‌ها) Error! Bookmark not defined.

۴. جادوی ابعاد: ماتریس‌ها Error! Bookmark not defined.

۵. جعبه ابزار جادویی: ۱۰ طلسم پرکاربرد Error! Bookmark not defined.

۶. مسابقه بزرگ: مفهوم زمان‌سنجی Error! Bookmark not defined.

۷. پروژه: چالش ۱,۰۰۰,۰۰۰ ماجراجو Error! Bookmark not defined.

فصل ۱۹: کتابخانه بزرگ پانداس (رام کردن داده‌های جدولی)
Error! Bookmark not defined.

۱. احضار Pandas Error! Bookmark not defined.

۲. DataFrame چیست؟ (جدول جادویی) Error! Bookmark not defined.

۳. خواندن طومارهای باستانی (فایل‌های CSV) Error! Bookmark not defined.

۴. ذره‌بین جادویی: فیلتر کردن داده‌ها Error! Bookmark not defined.

۵. جعبه ابزار پانداس: ۱۰ طلسم پرکاربرد Error! Bookmark not defined.

۶. پروژه: تحلیل تابلوی امتیازات Error! Bookmark not defined.

فصل ۲۰: نقشه‌ی مسحور (روایت داستان با Matplotlib) Error! Bookmark not defined.

۱. احضار نقاش سلطنتی (Matplotlib) Error! Bookmark not defined.

۲. اولین قلم‌مو (plot) - نمودار خطی Error! Bookmark not defined.

۳. ستون‌های مقایسه (bar) - نمودار میله‌ای Error! Bookmark not defined.

۴. زیبایی‌شناسی جادویی (شخصی‌سازی) Error! Bookmark not defined.

۵. جعبه ابزار نقاش: ۱۰ طلسم پرکاربرد Error! Bookmark not defined.

۶. پروژه: گزارش تصویری انجمن Error! Bookmark not defined.

بخش هفتم: ماجراجویی ادامه دارد...

فصل ۲۱: افق‌های بی‌پایان (نقشه‌های گنج آینده) Error! Bookmark not defined.

۱. قدم‌های بعدی (The Essential Next Steps) Error! Bookmark not defined.

۲. مسیر معماران دیجیتال (Programming & Web) Error! Bookmark not defined.

۳. مسیر پیشگویان داده (AI & Data) Error! Bookmark not defined.

۴. مسیر محافظان و مدیران سیستم (SysAdmin & Security) Error! Bookmark not defined.

سخن پایانی: پایان آغاز Error! Bookmark not defined.

منابع و مآخذ Error! Bookmark not defined.

بخش اول: شروع ماجرای جویی!

در این بخش، مشعل را روشن می‌کنیم، با دنیای کد آشنا می‌شویم و یاد می‌گیریم چطور مثل یک ماجراجوی حرفه‌ای، اولین معماها را حل کنیم.

فصل ۱: اولین سفر:

آماده‌سازی محیط توسعه

مأموریت فصل : کارت معرفی برای ورود به انجمن ماجراجویان
نقشه راه:

- چرا پایتون؟
- آماده کردن کوله‌پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین
- اولین ورد جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!
- پروژه: ساخت کارت معرفی مخصوص ماجراجویان با نام خودتان.

چرا پایتون؟

صدها زبان برنامه‌نویسی وجود دارد، پس چرا همه دارن از پایتون حرف می‌زنن و چرا ما این ماجراجویی رو با پایتون شروع می‌کنیم؟

۱. یادگیری مثل چت کردنه، خیلی راحت!

بزرگ‌ترین فرق پایتون با زبان‌هایی مثل C++ یا Java اینه که سینتکس (قواعد نوشتاری) خیلی ساده و روانی داره. انگار داری به زبان انگلیسی دستور می‌دی. به جای اینکه درگیر کلی علامت عجیب و غریب و قوانین سخت بشی، مستقیم میری سر اصل مطلب و خلاقیت رو پیاده می‌کنی.

۲. با یک تیر، سه تا نشونه رو هدف می‌گیری!

پایتون فقط برای شروع کردن عالی نیست؛ اونقدر قدرتمنده که بزرگترین شرکت‌های دنیا دارن ازش برای ساختن آینده استفاده می‌کنن. با یادگیری پایتون، در واقع کلید ورود به سه تا از خفن‌ترین دنیاهای تکنولوژی رو به دست میاری:

هوش مصنوعی (AI): فکر کردی جادوهایی مثل ChatGPT چطور یاد می‌گیرن باهات حرف بزنن، یا ماشین‌های خودران چطور رانندگی رو یاد می‌گیرن؟ ورد اصلی برای آموزش دادن به این هوش‌های مصنوعی، پایتونه. یعنی جادوگرهای دنیای تکنولوژی، با استفاده از پایتون به این ماشین‌ها یاد میدن که چطور فکر کنن، الگوها رو تشخیص بدن و تصمیم بگیرن. در واقع، پایتون زبان اصلی برای ساختن و تربیت کردن مغز متفکریه که پشت این تکنولوژی‌های شگفت‌انگیز قرار داره. (البته زبان‌های دیگه ای هم هستن ولی پایتون برای آموزش هوش مصنوعی پر طرفدار ترینه)

توسعه وب (Web Development): اون بخش‌های پنهان و مغز متفکر سایت‌ها و اپلیکیشن‌های معروفی مثل اینستاگرام، یوتیوب و کافه بازار با پایتون ساخته شده. (البته سایت‌های بزرگ از زبان‌های دیگه هم استفاده کردن و فقط پایون به تنهایی نیست)

علم داده (Data Science): امروز، داده‌ها مثل گنج‌های پنهان هستن. پایتون بهترین بیل و کلنگ برای کشف این گنج‌هاست. باهاش می‌تونن حجم بالایی از اطلاعات رو تحلیل کنی، الگوهای عجیب و غریب پیدا کنی و آینده رو پیش‌بینی کنی!

حرف آخر:

انتخاب پایتون مثل انتخاب یک چاقوی سوئیسی برای شروع ماجراجویییه. ، تو با یک ابزار ساده، قدرتمند و همه‌کاره شروع می‌کنی که هم راه رو برات باز می‌کنه و هم تو رو به هر مقصدی که بخوای می‌رسونه.

پس کوله‌پشتیت رو آماده کن، چون قراره با این زبان جادویی، اولین قدم‌ها رو برای تبدیل شدن به یک ماجراجوی واقعی در دنیای تکنولوژی برداریم! 🚀

آماده کردن کوله‌پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین

هر ماجراجویی به ابزار نیاز داره. ابزار اصلی ما، خودِ زبان پایتونه. برای استفاده از قدرت پایتون، دو راه اصلی پیش روی ماست: یکی اینکه **کارگاه شخصی** خودمون رو بسازیم (نصب پایتون روی کامپیوتر و یا لپ‌تاپ خودمون) و دومی اینکه از یک **آزمایشگاه آنلاین** جادویی و آماده استفاده کنیم (پایتونی که به صورت آنلاین ارائه میشه مناسب بچه‌هایی هست که سیستم قوی ندارن و یا از گوشی و تبلت استفاده میکنن).

انتخاب با توه، ماجراجو!

مسیر اول: ساخت کارگاه شخصی (نصب پایتون روی کامپیوتر و لپ‌تاپ)

این مسیر برای کسانی که دوست دارند همه چیز تحت کنترل خودشان باشد و ابزارهاشان رو روی سیستم شخصی‌شان داشته باشند.

قدم اول: نصب مترجم پایتون

۱. برای کاربران ویندوز (Windows)

به وب‌سایت رسمی پایتون python.org برو. اونجا آخرین نسخه پایتون منتظرته. روی دکمه "Downloads" کلیک کن و فایل نصب (installer) رو برای ویندوز دانلود کن.

مهم‌ترین نکته نصب: فایل نصبی رو که اجرا کردی، به یک صفحه نصب می‌رسی. خیلی خیلی مهمه که در همون صفحه اول، تیک گزینه‌ی **Add Python to PATH** رو بزنی. این کار مثل اینه که به ویندوز بگی "هی، من یه ابزار جدید به اسم پایتون دارم، حواست بهش باشه. (اگه این کار رو انجام ندی زمانی که کدهای پایتون رو اجرا کنی بهت ارور اینکه پایتون رو نمیشناسم میده)

نصب نهایی: حالا روی **Install Now** کلیک کن و منتظر بمون تا جادو کامل بشه. تمام! کارگاه شخصی تو آماده‌ست.

۲. برای کاربران مک (The Mac Guild)

کاربران مک هم می‌توانند به سادگی از نصب‌کننده رسمی پایتون استفاده کنند تا جدیدترین ابزارها را در اختیار داشته باشند.

سفر به منبع اصلی: به وب‌سایت رسمی پایتون python.org برو و وارد بخش "Downloads" شو. وب‌سایت به طور هوشمند سیستم عامل مک تو را تشخیص می‌دهد. دریافت ابزار: روی دکمه دانلود آخرین نسخه برای macOS کلیک کن. یک فایل با پسوند **.pkg** یا **.dmg** خواهد شد.

اجرای جادوی نصب: فایل نصب کننده را باز کن. این کار یک پنجره نصب استاندارد مک را باز می‌کند. مراحل بسیار ساده است؛ فقط کافیست روی دکمه‌های Continue, Agree و در نهایت Install کلیک کنی. در این مرحله، مک از تو رمز عبور سیستم را می‌خواهد تا اجازه نصب را صادر کند.

پایان نصب: پس از چند لحظه، نصب‌کننده کارش را تمام می‌کند و پایتون به صورت کامل روی سیستم تو نصب می‌شود. حالا کارگاه شخصی تو روی مک هم آماده است!

۳. برای کاربران لینوکس (Linux)

لینوکس، سرزمین موردعلاقه کدنویس‌هاست و پایتون معمولاً جزئی از خون این سیستم‌عامله. اما باید مطمئن بشیم که جدیدترین نسخه رو داریم. چک کردن نسخه: ترمینال رو باز کن و این دستور رو تایپ کن:

```
python3 --version
```

اگه نسخه‌ای که نشون می‌ده جدیده (مثلاً ۳.۶ به بالا)، کارت راحته! آپدیت کردن ابزار: اگه نسخه‌ت قدیمیه یا اصلاً نصب نیست، با توجه به توزیع لینوکست (مثل اوبونتو، فدورا و...)، با یک دستور ساده آپدیتش کن. برای اکثر سیستم‌های مبتنی بر دیبیا (مثل اوبونتو)، این دستور کافیه:

```
sudo apt update  
sudo apt install python3
```

تمام! تو آماده‌ای.

قدم دوم: آماده کردن دفترچه جادو (محیط توسعه VS Code)

حالا که پایتون را نصب کردیم، به یک "دفترچه جادویی" برای نوشتن وردها و طلسم‌هایمان (کدها) نیاز داریم. بهترین انتخاب برای این کار، Visual Studio Code (یا به اختصار VS Code) است.

فکر کن VS Code یک دفترچه خفنه که کلمات جادویی رو برات رنگی می‌کنه، اگه وردی رو اشتباه بنویسی زیرش خط می‌کشه و کمک می‌کنه کدهات رو مثل یک ماجراجوی حرفه‌ای، تمیز و مرتب نگه داری.

دانلود دفترچه جادو: به وب‌سایت رسمی code.visualstudio.com برو. سایت به طور خودکار سیستم عامل تو (ویندوز، مک یا لینوکس) را تشخیص می‌دهد. روی دکمه بزرگ دانلود کلیک کن.

نصب ساده: فایل دانلود شده را اجرا کن و مراحل نصب را که بسیار ساده است، دنبال کن.

قدم سوم: افزونه شناسایی پایتون در دفترچه جادویی (VsCode)

برای اینکه دفترچه جادویی ما زبان پایتون را به بهترین شکل بفهمد، باید یک افزونه (Extension) به آن اضافه کنیم:

VS Code را باز کن. از نوار کناری سمت چپ، روی آیکونی که شبیه چند مربع است کلیک کن (بخش Extensions). در کادر جستجو، کلمه Python را تایپ کن.

اولین گزینه‌ای که توسط Microsoft ارائه شده را پیدا کن و روی دکمه آبی رنگ Install کلیک کن.

تبریک! حالا کارگاه شخصی تو با بهترین ابزارها آماده‌ی ماجراجویی است.

مسیر دوم: استفاده از آزمایشگاه جادویی آنلاین (Google Colab)

اگه حوصله نصب و درگیری با سیستم عامل رو نداری، یا با گوشی و تبلت این ماجراجویی رو دنبال می‌کنی، یا سیستم خیلی قوی نیست، این مسیر برای تو ساخته شده!

گوگل کولب (Google Colab) یک دفترچه یادداشت آنلاین و هوشمند که پایتون از قبل روش نصب شده و فقط منتظر دستورات جادویی توه.

چرا خفنه؟

بدون نیاز به نصب: فقط با یک مرورگر و اکانت گوگل (رایگان) کارت راه میفته.

همیشه در دسترس: از هرجایی (کافه، کتابخونه، اتوبوس) و با هر وسیله‌ای (لپ‌تاپ، تبلت، گوشی) به کدهات دسترسی داری.

قدرت ابری: انگار داری با کامپیوترهای قدرتمند گوگل کار می‌کنی، پس نگران ضعیف بودن سیستم نباش. (در عمل کدهات روی سرورهای گوگل اجرا میشه)

چطور شروع کنیم؟

فقط کافیه به وبسایت `colab.research.google.com` بری، با اکانت گوگل وارد بشی و روی New notebook کلیک کنی. یک صفحه جدید باز می‌شه که می‌تونی اولین کدهای پایتون رو همونجا بنویسی و اجرا کنی. به همین سادگی!

خب ماجراجو، کوله‌پشتیت رو انتخاب کردی؟ ابزارت رو آماده کن که می‌خوایم اولین ورد جادویی رو اجرا کنیم!

اولین ورد جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!

تبریک ماجراجو! کوله‌پشتیات آماده است و ابزارهایت را انتخاب کرده‌ای. اکنون زمان آن رسیده که اولین ورد جادویی خود را یاد بگیری و اولین طلسم را در دنیای کدنویسی اجرا کنی. این لحظه، لحظه‌ای فراموش‌نشدنی است؛ لحظه‌ای که برای اولین بار به ماشین دستور می‌دهی و او به تو پاسخ می‌دهد.

این ورد جادویی، `print` نام دارد. کارش خیلی ساده است: هر چیزی که تو داخل پرانتز و بین دو علامت نقل قول (" ") به او بدهی، روی صفحه نمایش حک می‌کند. (مثل `print("Hi")`)

بیایید با هم این طلسم را اجرا کنیم!

اگر از مسیر اول (کارگاه شخصی و VS Code) می‌آیی: (برای بچه‌های `colab` بخش جدایی رو در ادامه توضیح میدم)

برای اینکه ماجراجویی‌مان مرتب باشد، اول یک پوشه (Folder) برای پروژه‌هایمان می‌سازیم.

ساخت پوشه پروژه: روی دسکتاپ یا هر جای دیگری که دوست داری، یک پوشه (Folder) جدید بساز و اسم آن را پروژه بگذار. تمام طلسم ها، گنج‌ها و نقشه‌های ما در این قلمرو نگهداری خواهد شد.

باز کردن دفترچه جادو (VS Code): برنامه VS Code را که قبلاً نصب کردی، باز کن. فراخوانی قلمرو: از منوی بالا، روی File کلیک کرده و گزینه‌ی Open Folder را انتخاب کن. حالا به همان پوشه‌ی پروژه که ساختی برو و آن را انتخاب کن. با این کار، VS Code تمام تمرکزش را روی قلمرو ماجراجویی تو می‌گذارد.

ساخت اولین فایل پایتون: در سمت چپ VS Code، جایی که اسم پوشه‌ی پروژه را می‌بینی، یک آیکون کوچک شبیه به یک برگه با علامت + وجود دارد (New File). روی آن کلیک کن و اسم فایل جدید را project.py بگذار. پسوند .py به کتاب جادوی ما می‌گوید که این یک طومار پایتون است! (حتماً پسوند فایل رو .py. بزار این خیلی مهمه! و اگه نزاری دیگه vs code فایلت رو درست نمیشناسه)

نوشتن ورد جادویی: حالا در صفحه‌ی سفیدی که باز شده، اولین ورد جادویی خود را با دقت تایپ کن (خیلی به علامت های " و فاصله ها و ... دقت کن چون توی برنامه نویسی اگه حتی یک فاصله رو اشتباه بزاری کدت کار نمیکنه):

```
1. print("Hello, World!")
```

اجرای طلسم! وقت هیجان‌انگیز ماجرا فرا رسیده. باید "ترمینال" یا همان اتاق فرمان را باز کنیم تا ورد خود را در آن بخوانیم.

از منوی بالای VS Code، روی Terminal کلیک کن و New Terminal را انتخاب کن. راه میان‌بر و سریع‌تر: می‌توانی دکمه‌های ترکیبی Ctrl + ~ (کلید ~ معمولاً کنار عدد ۱ روی کیبورد است) را فشار دهی تا ترمینال فوراً باز شود. در مک، این میان‌بر Cmd + ~ است.

خواندن دستور نهایی: در ترمینالی که پایین صفحه باز شده، این دستور را تایپ کن و کلید Enter را بزن (به فاصله و املائی کلمات دقت کن و همچنین اگر اسم فایل پایتون رو چیز دیگه ای گذاشتی اون رو اینجا بنویس و به صورت کامل با پسوند .py):

```
python project.py
```

(نکته برای ماجراجویان مک و لینوکس: بجای دستور بالا آن python3 project.py را امتحان کنید.)

و ناگهان... جادو اتفاق می‌افتد!

درست در خط بعدی ترمینال، این پیام ظاهر می‌شود:

```
Hello, World!
```

تبریک! تو همین الان اولین کد خود را اجرا کردی. تو به کامپیوتر دستور دادی و او از تو اطاعت کرد. این اولین قدم تو برای تبدیل شدن به یک جادوگر قدرتمند در دنیای کدنویسی است. این لحظه را جشن بگیر!

اگر از مسیر دوم (آزمایشگاه جادویی Google Colab) می‌آیی:

کار تو بسیار ساده‌تر است! آزمایشگاه آنلاین همه چیز را برایت آماده کرده.

ورود به آزمایشگاه: در همان دفترچه یادداشت جدیدی که در Colab باز کردی، یک سلول

کد خاکستری رنگ با یک دکمه "پلی" () در کنارش می‌بینی.

نوشتن ورد جادویی: داخل این سلول، ورد جادویی را تایپ کن (نکته اعداد در شروع هر

خط جزوی از کد نیستن و فقط راهنمایی ما هستن و اون هارو نباید تایپ کنی):

```
1. print("Hello, World!")
```

اجرای طلسم: حالا کافیه روی همان دکمه "پلی" () کلیک کنی یا کلیدهای Shift

+ Enter را با هم فشار دهی.

بلافاصله زیر سلول کد، نتیجه‌ی جادوی خود را خواهی دید:

Hello, World

فوق‌العاده بود، نه؟ تو اولین پیام خود را از دل ابرها به دنیا فرستادی! این قدرت آزمایشگاه جادویی است. (منظور از ابرها سرورهای ابری گوگل هست) ماجراجویی رسماً آغاز شد! تو اولین طلسم را با موفقیت اجرا کردی. حالا بریم یک پروژه باحال باهم بزنیم!

پروژه: ساخت کارت ویزیت دیجیتال

ماجرای جوی عزیز، تو اولین ورد جادویی را با موفقیت اجرا کردی و به دنیا سلام دادی! حالا وقت آن است که با استفاده از همین طلسم قدرتمند، هویت خودت را به "انجمن ماجراجویان پایتون" اعلام کنی. ما یک کارت ویزیت دیجیتال خواهیم ساخت تا همه بدانند با چه قهرمان جدیدی آشنا شده‌اند.

قبل از ساختن کارت، بیا کمی عمیق‌تر به جادویی که استفاده کردیم نگاه کنیم.

ورد `print()` و علامت‌های نقل قول (" ") برای چیست؟

`print()`: فرمان نمایش

فکر کن `print()` یک فرمان مستقیم به کامپیوتر است. وقتی می‌گویی `print`، در واقع داری به او دستور می‌دهی: "هی، چیزی که بهت می‌گم رو بگیر و روی صفحه نمایش بده!". این یک طلسم برای نمایش دادن است.

" ": مرز دنیای جادو و دنیای واقعی

پایتون باید بداند که حرف‌های تو دقیقاً چیست. علامت نقل قول (" ") یک مرز جادویی ایجاد می‌کند. هر چیزی که داخل این علامت‌ها باشد محتوای متنی است و یک "رشته" (String) نامیده می‌شود. پایتون به محتوای داخل رشته‌ها دست نمی‌زند و آن را دقیقاً

همانطور که هست، نمایش می‌دهد. اگر این علامت‌ها را نگذاریم، پایتون فکر می‌کند که کلمات داخل پرانتز، وردهای جادویی دیگری هستند و گیج می‌شود!

پایتون چگونه کدهای ما را می‌خواند؟ مثل یک آبشار!

یک نکته خیلی مهم در مورد پایتون این است که کدها را مثل یک آبشار، از بالا به پایین می‌خواند و اجرا می‌کند. او از خط اول شروع می‌کند، دستورش را اجرا می‌کند، سپس به خط دوم می‌رود، آن را اجرا می‌کند و همینطور تا آخر ادامه می‌دهد. این ترتیب خیلی مهم است، چون به ما اجازه می‌دهد داستان ماجراجویی خود را مرحله به مرحله تعریف کنیم.

در هر ماجراجویی، یادداشت‌برداری برای به خاطر سپردن نکات مهم ضروری است. در دنیای کدنویسی، ما این یادداشت‌ها را با استفاده از علامت # (هشتگ) می‌نویسیم. هر چیزی که بعد از علامت # در یک خط نوشته شود، "کامنت" نام دارد. پایتون به طور کامل کامنت‌ها را نادیده می‌گیرد و آن‌ها را اجرا نمی‌کند. کامنت‌ها فقط برای ما ماجراجویان هستند تا به خودمان (یا به هم‌تیمی‌هایمان) یادآوری کنیم که یک طلسم (کد) خاص چه کاری انجام می‌دهد، یا چرا آن را به این شکل نوشته‌ایم. فکر کن داری در حاشیه دفترچه جادوی خودت، نکات مهم را یادداشت می‌کنی تا بعداً فراموششان نکنی!

- | |
|------------------------------------|
| 1. # some text
2. # some text 2 |
|------------------------------------|

آماده برای ساخت کارت ویزیت؟

حالا که رازهای `print()` و ترتیب اجرای کدها را می‌دانی، بیا کارت ویزیت خودت را بسازیم. یادت باشد که اطلاعات داخل علامت‌های نقل قول را با مشخصات خودت عوض کنی!

اجرا کن و نتیجه را ببین! (به علامت پرانتز و " و # خیلی دقت کن!)

کد پروژه در `project.py` بنویس (یا در یک سلول در Colab):

برای یادگیری بهتر سعی کن یکبار از روی این کد بنویسی و یکبار بدون نگاه کردن به این کد اون رو بنویسی!

```
1. # --- Adventurer's ID Card ---
2.
3. # Line 1: Displays the adventurer's name
4. print("Name: [Enter your name here]")
5.
6. # Line 2: Your chosen title
7. print("Title: Python World Explorer")
8.
9. # Line 3: A way for others to contact you
10. print("Contact: adventurer@example.com")
11.
12. # Line 4: Your personal motto for this journey
13. print("Motto: Ready to solve the first puzzle!")
14.
15. print("=====")
```

وقتی این کد را اجرا کنی (با دستور `python project.py` در ترمینال یا دکمه  در Colab)، می‌بینی که پایتون دقیقاً به همان ترتیبی که نوشته‌ای، خط به خط اطلاعات تو را چاپ می‌کند و کارت ویزیت دیجیتال تو را می‌سازد.

آفرین! تو نه تنها اولین کد خود را اجرا کردی، بلکه اولین پروژه عملی خود را هم به پایان رساندی. این کارت ویزیت، مدرک ورود تو به دنیای شگفت‌انگیز کدنویسی است. ماجراجویی تازه شروع شده!

کارتت رو توی شبکه‌های اجتماعی با هشتگ `#codewithmahdi` با ما به اشتراک بزار.

توی فصل بعد باهم یاد می‌گیریم کارهای باحال‌تری انجام بد.

فصل ۲: تبدیل شدن به کارآگاه کد

مأموریت فصل: شکار اولین "باگ".

نقشه راه:

- ذهنیت یک ماجراجو: برنامه نویسی یعنی حل معما، نه حفظ کردن ورد!
- جعبه ابزار کارآگاه:
- Stack Overflow: کتابخانه بزرگ دانش تمام جادوگران دنیا.
- دستیارهای هوشمند (AI): چطور از غول های چراغ جادو Gemini, ChatGPT کمک بگیریم؟
- هنر پرسیدن سوال خوب.
- پروژه: یک کد نفرین شده به شما داده می شود . با ابزارهای جدیدتان، خطا را پیدا و آن را فیکس کنید.

ذهنیت یک ماجراجو: حل معما، نه صرفاً حفظ کردن ورد!

اولین و مهم‌ترین قانونی که هر ماجراجوی کهنه‌کاری می‌داند این است:

برنامه‌نویسی یعنی حل معما، نه صرفاً حفظ کردن ورد.

بسیاری از کسانی که تازیع شروع میکنند فکر میکنند که باید صدها دستور و طلسم جادویی مانند print را «حفظ» کنند. اما جادوی واقعی در این نیست. جادوی واقعی در «تفکر منطقی» توست.

وقتی کد تو کار نمی‌کند یا با یک پیام خطای ترسناک قرمز رنگ (که به زودی با آن‌ها آشنا می‌شویم) مواجه می‌شوی، نترس! این یک معما است. کامپیوتر، برخلاف انسان‌ها، موجودی کاملاً منطقی است. او دقیقاً کاری را می‌کند که به او می‌گویی. اگر نتیجه اشتباه است، یعنی ما دستور را به شکلی نامفهوم یا اشتباه به او داده‌ایم.

به این دستورات اشتباه، «باگ» یا «طلسم معیوب» می‌گویند. و مأموریت ما در این فصل، پیدا کردن (Debugging) یا «رفع طلسم» کردن آن‌هاست.

پس ذهنیت خود را آماده کن: تو یک کارآگاه هستی. هر خطا (Error) یک سرخ است. هر باگ، یک چالش جذاب است که منتظر توست تا آن را حل کنی.

جعبه ابزار کارآگاه

هر کارآگاهی به ابزار نیاز دارد. برای شکار باگ‌ها، لازم نیست همه چیز را از صفر اختراع کنیم. ما دو ابزار فوق‌العاده قدرتمند در «کوله پشتی» خود داریم که باید طرز استفاده از آن‌ها را یاد بگیریم.

۱. Stack Overflow: کتابخانه بزرگ دانش تمام جادوگران دنیا

تصور کن یک کتابخانه جادویی بی‌انتهای وجود دارد که در آن، هر جادوگر و ماجراجویی که تا به حال با یک طلسم معیوب روبرو شده، مشکل و راه‌حلش را در آن ثبت کرده است.

آن کتابخانه، Stack Overflow است.

این یک وبسایت پرسش و پاسخ برای برنامه‌نویسان است. به احتمال ۹۹٪، هر مشکلی که تو در این ماجراجویی با آن روبرو می‌شوی، قبلاً صدها نفر دیگر هم با آن روبرو شده‌اند و بهترین جادوگران دنیا به آن پاسخ داده‌اند. یادگیری «جستجو کردن» در این کتابخانه، اغلب مهم‌تر از حفظ کردن هر وردی است. (برای استفاده کافی هست پیغام خطایی که با آن مواجه شدی رو در گوگل سرچ کنی و حتماً از بین نتیجه جستجو Stack Overflow رو میبینی)

۲. دستیارهای هوشمند (AI): غول‌های چراغ جادوی

حالا تصور کن غول‌هایی وجود دارند که نه تنها دستورات ما را اجرا می‌کنند، بلکه می‌توانند به ما در نوشتن طلسم‌های بهتر کمک کنند، کدهای ما را توضیح دهند و حتی باگ‌های آن را پیدا کنند!

ابزارهایی مثل Gemini و ChatGPT دقیقاً همین کار را می‌کنند. آن‌ها دستیارهای هوشمند تو هستند. می‌توانی کد معیوب خود را به آن‌ها نشان دهی و بپرسی: «کجای این طلسم اشتباه است؟» آن‌ها با صبر و حوصله به تو کمک خواهند کرد تا سرنخ را پیدا کنی.

هنر پرسیدن سوال خوب

داشتن جعبه ابزار کافی نیست؛ باید بلد باشی چطور از آن استفاده کنی. چه در Stack Overflow و چه از یک دستیار هوشمند، «خوب پرسیدن» یک هنر است. یک ماجراجوی تازه‌کار فریاد می‌زند: «کمک! طلسم کار نمی‌کند!» و هیچ‌کس نمی‌تواند به او کمک کند.

اما یک کارآگاه حرفه‌ای می‌گوید: «من در حال اجرای یک طلسم print در فایل project.py هستم. انتظار داشتم کارت ویزیت دوستم را چاپ کنم، اما در عوض با این پیام خطای قرمز مواجه شدم: SyntaxError: EOL while scanning string literal. این هم کدی که نوشته‌ام. به نظرتان مشکل از کجاست؟»

همیشه سه چیز را در سوالت بیاور:

۱. چه کار می‌خواستی بکنی؟ (هدف تو چه بود؟)
 ۲. چه مشکلی پیش آمد؟ (پیام خطا دقیقاً چه بود؟ آن را کپی کن.)
 ۳. کدی که نوشتی چه بود؟ (کد خودت را نشان بده.)
- با این سه سرنخ، هر کسی می‌تواند به تو در شکار باگ کمک کند.

پروژه: شکار باگ در کد نفرین شده

آماده‌ای اولین شکار را انجام دهی، کارآگاه؟

در زیر، یک «کد پایتون» به تو داده شده که قرار بود «کارت ویزیت» ماجراجوی دیگری را چاپ کند، اما طلسم آن توسط یک باگ معیوب شده است. آن را در فایل `project.py` خودت (یا در یک سلول جدید در Google Colab) کپی کن و سعی کن آن را اجرا کنی.

کد طلسم‌شده:

```
1. # The Cursed ID Card
2. print("Name: Mahdi ghaemi")
3. print("Title: World Ender")
4. print("Motto: "Brave and Bold!")
5. print(=====)
```

حالا، کلاه کارآگاهی‌ات را بگذار و شروع کن:

۱. اجرای طلسم: کد را اجرا کن. به پیام خطای قرمزی که پایتون به تو نشان می‌دهد در «ترمینال» (یا زیر سلول Colab) به دقت نگاه کن. این اولین سرنخ توست!
۲. پیدا کردن سرنخ: پایتون معمولاً به تو می‌گوید که در کدام خط (line) به مشکل خورده است. به آن خط برو.
۳. بررسی مژگون‌ها: با استفاده از دانشی که از فصل ۱ داری (به خصوص در مورد ورد `print` و «مرزهای جادویی» "")، سعی کن مشکل را پیدا کنی. آیا همه‌ی پرانتزها باز و بسته شده‌اند؟ آیا همه‌ی «مرزهای جادویی» (علامت‌های نقل قول) به درستی جفت شده‌اند؟

۴. استفاده از جعبه ابزار: اگر گیر کردی، از دستیار هوشمندت (ChatGPT) بپرس یا کلمات کلیدی پیام خطا (مثلاً `SyntaxError`) را در «کتابخانه» `Stack Overflow` جستجو کن. وقتی توانستی کد را طوری «رفع طلسم» کنی که کارت ویزیت زیر به درستی و بدون هیچ پیام خطای قرمزی چاپ شود، تو رسماً اولین باگ خود را شکار کرده‌ای!

نتیجه نهایی (کد رفع نفرین شده):

```
1. # The Fixed ID Card
2. print("Name: Mahdi ghaemi")
3. print("Title: World Ender")
4. print("Motto: Brave and Bold!")
5. print("=====")
```

تبریک می‌گویم، کارآگاه! تو با موفقیت از ذهنیت حل مسئله و ابزارهایت استفاده کردی. «کوله پشتی» تو برای ماجراجویی بعدی سنگین‌تر و ارزشمندتر شد. امیدوارم قدر این جعبه ابزار رو بدونی چون در گذشته به این خوبی وجود نداشتند! در فصل بعد، ابزارهای جادویی جدیدی برای «تقسیم غنائم» (کار با اعداد) خواهیم ساخت!

فصل ۳: جعبه های جادویی:

ماشین حساب جادویی

مأموریت فصل: ساخت یک ماشین حساب جادویی برای محاسبه و تقسیم غنائم بین اعضای گروه

نقشه راه

- جعبه های جادویی (متغیرها): چطور غنائم و اطلاعات را در حافظه نگه داریم؟
- آشنایی با غنائم: انواع داده های پایه (Integers, Floats, Strings, Booleans).
- جادوی ریاضی: کار با عملگرهای + ، - ، * و / روی اعداد.
- جادوی متن: چطور طلسم + برای متن ها (Strings) متفاوت عمل می کند؟
- پروژه: ساخت یک ماشین حساب که غنائم (اعداد ثابت) را محاسبه و نتایج را چاپ می کند.

۱. جعبه‌های جادویی (متغیرها)

وقتی در ماجراجویی خود یک «سکه طلا» یا یک «معجون سلامتی» پیدا می‌کنی، آن را روی زمین رها نمی‌کنی تا گم شود! تو آن را در «کوله پشتی» یا یک «جعبه» امن می‌گذاری تا بعداً از آن استفاده کنی.

در برنامه‌نویسی، به این جعبه‌های جادویی «متغیر» (Variable) می‌گوییم.

متغیرها، مکان‌هایی در حافظه (RAM) کامپیوتر هستند که ما داده‌هایمان را در آن‌ها ذخیره می‌کنیم. این کار باعث می‌شود بتوانیم بعداً به آن داده‌ها دسترسی داشته باشیم. این داده‌ها موقتی هستند و وقتی برنامه تمام شود (یا کامپیوتر خاموش شود)، از بین می‌روند. (در فصل‌های بعد با اینکه چجوری به صورت دائمی اطلاعات رو ذخیره کنیم بیشتر آشنا میشیم)

ساختن یک متغیر مثل برچسب زدن به یک جعبه است. ما از علامت = (که به آن عملگر تخصیص می‌گوییم) برای ریختن چیزی در جعبه استفاده می‌کنیم (یادت نره وقتی # رو اول یه خطی می‌ذاریم یعنی اون خط توضیحیه و پایتون اون رو اجرا نمی‌کنه):

```
1. # "Mahdi Ghaemi" is the content inside the box
2. # adventurer_name is a label
3. adventurer_name = "Mahdi Ghaemi"
4.
5. # gold_coins is a label
6. # 50 is the content inside the box
7. gold_coins = 50
```

حالا، هر وقت که بخواهیم، می‌توانیم محتوای داخل جعبه را ببینیم. از `print` و نام اون جعبه استفاده می‌کنیم (دقت کن که نام متغیر نباید داخل " باشه وگرنه با متن اشتباه گرفته میشه):

```
1. # Let's make some magic boxes
2. adventurer_name = "Mahdi Ghaemi"
3. gold_coins = 50
4. health_points = 100
5.
6. # Now let's print the content of the boxes
7. print("Adventurer's Name:")
8. print(adventurer_name) # Python prints the box content, not its name
9.
10. print("Gold Coins:")
11. print(gold_coins)
12.
```

```
13. print("Health Points:")
14. print(health_points)
```

اجرا کن و نتیجه را ببین! وقتی این کد را اجرا کنی، کامپیوتر به جای چاپ کلمه "gold_coins"، می‌رود و محتوای داخل آن جعبه (یعنی ۵۰) را چاپ می‌کند.

قدرت متغیرها: تغییرپذیری

جادوی واقعی متغیرها این است که محتوای آن‌ها می‌تواند تغییر کند.

```
1. # At the start of the adventure, we have 10 coins
2. gold_coins = 10
3. print("Current gold coins:")
4. print(gold_coins)
5.
6. # We defeat a giant and get 20 new coins
7. # We replace the old content of the box with the new value
8. gold_coins = 20
9. print("Gold coins after defeating the giant:")
10. print(gold_coins)
11.
12. # We can even do calculations on the box itself
13. # We spend 5 coins
14. gold_coins = gold_coins - 5
15. print("Remaining gold coins:")
16. print(gold_coins) # Output will be 15
```

این قابلیت برای ماجراجویی ما حیاتی است!

۲. آشنایی با غنائم: انواع داده‌های پایه (Primitive Types)

«جعبه‌های» جادویی تو (متغیرها) می‌توانند انواع مختلفی از «غنائم» (داده‌ها) را در خود نگه دارند. اما یک قانون بزرگ در این ماجراجویی وجود دارد: نوع غنیمت، کاربرد آن را مشخص می‌کند.

فکرش را بکن در یک ماجراجویی واقعی، تو نمی‌توانی یک «شمشیر» (که مثل متن است) را به «سکه‌های طلا» (که مثل عدد است) «اضافه» کنی. تو نمی‌توانی یک «طلسم جادویی»

(متن) را بر «تعداد اعضای گروه» (عدد) تقسیم کنی. هر غنیمتی، جادوی خاص خودش را دارد.

به این غنائم اصلی، «داده‌های پایه» (Primitive Types) می‌گویند. این‌ها بنیادی‌ترین مقادیری هستند که برای نمایش داده‌های اساسی به کار می‌روند و در خود زبان پایتون «ساخته» شده‌اند. پایتون ذاتاً می‌فهمد که «سکه» (عدد) چیست و «طومار» (متن) چیست و لازم نیست تو این‌ها را به او یاد بدهی.

در ماجراجویی پایتون، ما با چهار نوع غنیمت اصلی سروکار داریم:

اعداد صحیح (Integer)

این‌ها اعداد کامل و صحیح هستند. می‌توانند مثبت، منفی یا صفر باشند. هر وقت چیزی را می‌شماری (تعداد تیر، تعداد دشمن، تعداد قدم) از این نوع استفاده می‌کنی.

```
1. age = 21
2. score = -20
3. count = 0
```

اعداد اعشاری (Float)

این‌ها اعداد اعشاری هستند؛ مثل معجون‌هایی که می‌توانند خرد شوند. هر وقت به «دقت» نیاز داری (مثل محاسبه میزان سم یا درصد سلامتی) از این نوع استفاده می‌کنی.

```
1. pi = 3.14
2. temp = -273.0
3. percent = 99.9
```

رشته (String)

این‌ها «متن» هستند. هر چیزی که داخل علامت نقل قول (" یا ') قرار بگیرد، یک متن یا String است. نام‌ها، پیام‌ها، وردها و آدرس‌ها همه از این نوع هستند.

```
1. name = "mahdi"
2. message = "Oppsi"
```

بولین (Boolean)

این‌ها فقط دو حالت دارند: «روشن» یا «خاموش». True (درست) یا False (نادرست). این کلیدها برای تصمیم‌گیری استفاده می‌شوند. (آیا در باز است؟ True. آیا غول زنده است؟ False). ما در فصل بعد خیلی بیشتر از این کلیدها استفاده خواهیم کرد.

```
1. is_up = True
2. is_down = False
```

۳. جادوی ریاضی: کار با عملگرها

حالا که «عدد صحیح» (Integers) و «عدد اعشاری» (Floats) را می‌شناسیم، بیا با آن‌ها محاسبات انجام دهیم. پایتون چهار عملگر اصلی ریاضی را بلد است:

+ (جمع)

- (تفریق)

* (ضرب)

/ (تقسیم)

بیا این‌ها را با متغیرهایمان امتحان کنیم:

```
1. # We found the loot
2. gold_coins_chest1 = 50
3. gold_coins_chest2 = 75
4. adventurers_count = 4
5.
6. # --- Performing the magic of math ---
7.
8. # 1. Add the loot from both chests
9. total_gold = gold_coins_chest1 + gold_coins_chest2
10.
11. # 2. Divide the loot among the party members
```

```

12. gold_per_adventurer = total_gold / adventurers_count
13.
14. # --- Displaying the results ---
15. print("Total gold found:")
16. print(total_gold) # Output: 125
17.
18. print("Share per adventurer:")
19. print(gold_per_adventurer) # Output: 31.25

```

نکته مهم: دیدی که حاصل تقسیم ۱۲۵ (Integer) بر ۴ (Integer) شد ۳۱.۲۵ (Float)؟ در پایتون، عملگر / همیشه نتیجه را به صورت «گرد جادویی» (Float) برمی‌گرداند تا هیچ بخشی از غنائم در تقسیم از بین نرود!

۴. جادوی متن: چسباندن طومارها

یک نکته بسیار مهم! عملگر + یک جادوی دوگانه دارد.

وقتی با اعداد (Integers/Floats) استفاده شود، کار «جمع ریاضی» را انجام می‌دهد.

وقتی با متن‌ها (Strings) استفاده شود، کار «چسباندن» (Concatenation) را انجام می‌دهد.

این دو مثال را مقایسه کن:

```

1. # The magic of math (adding numbers)
2. num1 = 20
3. num2 = 5
4. result_math = num1 + num2
5. print("Result of math magic:")
6. print(result_math) # Output: 25
7.
8. # The magic of text (concatenating strings)
9. text1 = "20" # This is a string, not a number
10. text2 = "5" # This is also a string
11. result_text = text1 + text2
12. print("Result of text magic:")
13. print(result_text) # Output: "205"

```


این تفاوت بسیار مهم است و در فصل‌های آینده که می‌خواهیم ورودی بگیریم، دوباره با آن روبرو خواهیم شد. فعلاً فقط یادت باشد: پایتون بین «سکه» ۲۰ و «طومار» ۲۰ "تفاوت قائل می‌شود! (اولی عدد هست و دومی رشتست)

۵. پروژه: ساخت ماشین حساب جادویی

وقت آن است که تمام وردهای جادویی که امروز یاد گرفتیم (variables, primitive types و math operators) را با هم ترکیب کنیم تا ابزار نهایی این فصل را بسازیم. هدف: برنامه‌ای بنویس که دو عدد ثابت (که خودمان در کد به صورت متغیر می‌نویسیم) را بگیرد، هر چهار عمل اصلی (جمع، تفریق، ضرب، تقسیم) را روی آن‌ها انجام دهد و نتیجه را به زیبایی چاپ کند.

کد پروژه در **project.py** بنویس (یا در یک سلول در Colab):

برای یادگیری بهتر سعی کن یکبار از روی این کد بنویسی و یکبار بدون نگاه کردن به این کد اون رو بنویسی!

```
1. # --- Magic Calculator ---
2.
3. print("Welcome to the Magic Calculator!")
4. print("Today's numbers for calculation...")
5.
6. # --- Step 1: Define the loot (variables) ---
7. num1 = 150.5 # First treasure
8. num2 = 25.0 # Second treasure
9.
10. print("First number:")
11. print(num1)
12. print("Second number:")
13. print(num2)
14.
15. # --- Step 2: Perform math magic ---
16. sum_result = num1 + num2
17. subtract_result = num1 - num2
18. multiply_result = num1 * num2
19. divide_result = num1 / num2
20.
21. # --- Step 3: Display the results (Using + and str() casting) ---
```

```

22. print("=====")
23. print("Your magical calculation results:")
24.
26. print("Sum: ")
27. print(sum_result)
28.
29. print("Subtraction: ")
30. print(subtract_result)
31.
32. print("Multiplication: ")
33. print(multiply_result)
34.
35. print("Division: ")
36. print(divide_result)
37.
38. print("=====")

```

اجرا کن و نتیجه را ببین! خروجی تو باید یک ماشین حساب کامل و زیبا باشد:

نتایج جادویی محاسبات تو:

```

1. Welcome to the Magic Calculator!
2. Today's numbers for calculation...
3. First number:
4. 150.5
5. Second number:
6. 25.0
7. =====
8. Your magical calculation results:
9. Sum:
10. 175.5
11. Subtraction:
12. 125.5
13. Multiplication:
14. 3762.5
15. Division:
16. 6.02
17. =====

```

فصل ۴: منطق و شرطها:

ساخت بازی حدس عدد

مأموریت فصل: ساخت بازی "حدس عدد" و به چالش کشیدن کامپیوتر.

نقشه راه

- صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()`.
- تله‌ی کیمیاگری (تبدیل متن به عدد)
- دو راهی سرنوشت (`if` و `else`)
- مسیرهای چندگانه (`elif`)
- تکرار تا پیروزی (حلقه `while`)
- پروژه: پیاده‌سازی بازی حدس عدد.

۱. صحبت با ماجراجو: گرفتن ورودی از کاربر با ورود جادویی input()

تا الآن ما فقط با دستور print حرف می‌زدیم (اطلاعات می‌دادیم). حالا می‌خواهیم گوش کنیم (اطلاعات بگیریم). ورود جادویی برای شنیدن، input() است. وقتی پایتون به این خط می‌رسد، توقف می‌کند و منتظر می‌ماند تا شما چیزی تایپ کنید و دکمه Enter را بزنید.

```
1. print("what is your name?")
2. name = input()
3.
4. print("hi " + name + "weelcome")
```

نکته: می‌توانید پیام را داخل خود پرانتز input هم بنویسید تا کدتان کوتاه‌تر شود:

```
1. name = input("what is your name?")
2. print("hi " + name + "weelcome")
```

۲. تله‌ی کیمیاگری (تبدیل متن به عدد)

یک قانون خیلی مهم در پایتون وجود دارد: دستور input همیشه همه‌چیز را به صورت "متن" (String) می‌گیرد. حتی اگر شما عدد تایپ کنید!

اگر این کد را بنویسید، نتیجه فاجعه‌بار است:

```
1. # wrong code
2. num1 = input("num1:") # user enter -> 10
3. num2 = input("num2:") # user enter -> 20
4.
5. # python will "10" + "20" = "1020"
6. print(num1 + num2)
```

راه حل (کیمیاگری): ما باید متن را به عدد تبدیل کنیم تا پایتون به جای گذاشتن این ۲ عدد در کنار هم آن‌ها را با هم جمع کند. ورود جادویی برای تبدیل کردن به عدد صحیح، int() است.

```
1. # fixed code
2. text1 = input("num1:") # user enter -> 10
3. num1 = int(text1) # python will cast "10" to 10
```

```
4.
5. text2 = input("num2:") # user enter -> 20
6. num2 = int(text2) # python will cast "20" to 20
7.
8. print(num1 + num2) # 10 + 20 = 30
```

۳. دو راهی سرنوشت (if و else)

اینجا مهم‌ترین بخش منطق برنامه‌نویسی است. می‌خواهیم بگوییم: "اگر رمز درست بود، در را باز کن، وگرنه آژیر بکش." ما این کار را با if (اگر) و else (در غیر این صورت) انجام می‌دهیم.

قانون طلایی تورفتگی (Indentation)

پایتون چطور می‌فهمد کدام خط‌ها مربوط به "باز کردن در" است و کدام خط‌ها مربوط به "آژیر کشیدن"؟ جواب: با فاصله از سر خط.

هر وقت جلوی خطی علامت دو نقطه : گذاشتید (مثل آخر خط if)، خط‌های بعدی باید کمی جلوتر (معمولاً ۴ تا فاصله یا یک دکمه Tab و بهتر هست از Tab استفاده کنید) شروع شوند.

این فاصله به پایتون می‌گوید: "این خط‌ها متعلق به خط بالایی هستند." بیایید در عمل ببینیم:

```
1. password = "123"
2. guess = input("Enter password: ")
3.
4. if guess == password:
5.     # We have an indentation (Tab) here.
6.     # This block runs ONLY if the condition is True
7.     print("Access Granted: Open the door")
8.
9. else:
10.    # We have an indentation (Tab) here too.
11.    # This block runs if the condition is False (Wrong password).
12.    print("Access Denied: Enable alarm")
13.
14. # This line has NO indentation.
15. # It is outside the if/else blocks and always runs.
16. print("Continue program...")
```

تصور کنید: وقتی فاصله می‌دهید، انگار دارید دستورات را داخل یک جعبه می‌گذارید که روی آن نوشته if. اگر شرط درست نباشد، پایتون کل آن جعبه را دور می‌اندازد و اجرا نمی‌کند.

۴. مسیرهای چندگانه (elif)

اگر بخواهیم بیشتر از دو حالت داشته باشیم چه؟ مثلاً در بازی حدس عدد، فقط "درست" و "غلط" نداریم. سه حالت داریم:

- عدد درست است (برنده).
- عدد شما کوچک است (راهنمایی کن).
- عدد شما بزرگ است (راهنمایی کن).

برای این کار از elif (مخفف Else If) استفاده می‌کنیم.

```
1. secret_number = 7
2. user_guess = int(input("Guess a number: "))
3.
4. if user_guess == secret_number:
5.     print("Great! You guessed correctly.")
6.
7. elif user_guess < secret_number:
8.     print("Your number is too small!")
9.
10. else:
11.     # If it is not equal and not smaller, it must be larger.
12.     print("Your number is too big!")
```

نکته: پایتون به ترتیب از بالا چک می‌کند. به محض اینکه یکی از شرطها درست باشد، آن را اجرا می‌کند و بقیه را نادیده می‌گیرد.

۵. تکرار تا پیروزی (حلقه while)

مشکل کد بالا این است که فقط یک بار اجازه حدس زدن می دهد. ما می خواهیم بازی آنقدر تکرار شود تا بازیکن برنده شود.

ورد جادویی `while` (تا زمانی که) دقیقاً همین کار را می کند. ساختار آن دقیقاً مثل `if` است (نیاز به دو نقطه : و تورفتگی دارد)، با این تفاوت که وقتی کدهای داخلش تمام شد، دوباره به بالا برمی گردد و شرط را چک می کند.

```
1. # Create a variable to control the game state
2. game_over = False
3.
4. while game_over == False:
5.     # The lines below repeat constantly because they are indented
6.     print("The game is running...")
7.
8.     answer = input("Do you want to end the game? (yes/no) ")
9.
10.    if answer == "yes":
11.        game_over = True # This makes the loop condition False,
12.        # stopping the loop
13.    print("Goodbye!")
```

۶. پروژه: بازی حدس عدد

حالا بیا ببینیم تمام چیزهایی که یاد گرفتیم (`input`, `int`, `if/else`, `while`) را ترکیب کنیم و بازی را بسازیم.

کد پروژه در `project.py` بنویس (یا در یک سلول در Colab):

برای یادگیری بهتر سعی کن یکبار از روی این کد بنویسی و یکبار بدون نگاه کردن به این کد اون رو بنویسی!

```
1. # --- The Great Number Guessing Game ---
2.
3. # 1. Initial Setup
4. secret_number = 14 # The number the player must guess
5. correct_guess = False # No correct guess yet
6.
7. print("Welcome to the Number Guessing Game!")
8. print("I have chosen a number between 1 and 20.")
9.
10. # 2. Start the Loop
11. # Repeat WHILE the guess is not correct (correct_guess == False)
12. while correct_guess == False:
13.
14.     # Get input and convert to integer
15.     guess_text = input("What is your guess? ")
16.     guess = int(guess_text)
17.
18.     # 3. Check Game Logic (with precise indentation)
19.     if guess == secret_number:
20.         print("Excellent! You guessed correctly. The gate opens!")
21.         correct_guess = True # We flip the switch to end the loop
22.
23.     elif guess < secret_number:
24.         print("Your number is too small! Try higher.")
25.
26.     else:
27.         print("Your number is too big! Try lower.")
28.
29. # 4. End of Game
30. # This line has no indentation, meaning it is outside the while loop
31. print("Game Over. Thanks for trying!")
```

اجرا کن و نتیجه را ببین! کد را اجرا کن و سعی کن عدد جادویی را که خودت مخفی کرده‌ای پیدا کنی. بعداً میتوانی عدد ۱۴ را به هر عدد دیگری تغییر دهی و از دوستانانت بخواهی آن را حدس بزنند.

آفرین! تو همین الان به پایتون منطق آموختی. تو از یک آبشار ساده، به یک سیستم تصمیمگیری هوشمند رسیدی و اولین بازی کامل خودت را ساختی. کوله پشتی تو سنگینتر از همیشه است!